

Looker Studio to WhatsApp Report Bot

Implementation controls, risks and acceptance guide

Release 1.6.0 | Updated 5 October 2026

What changed

Since v1.1.0: durable retries with dead-lettering, dead letters that stop re-uploads and can be retried from the dashboard, one stored PDF per email, Signal key material kept out of logs, batched and cached WhatsApp key storage, LID chat IDs, and a Gmail bridge that checkpoints its progress and keeps the free Render service awake.

v1.6.0: anyone in a chat can use every report command for that chat; a chat dashboard at /chat/ logs in with a chat ID and manages only that chat (accepted risk: a chat ID is not a secret); WhatsApp is linked only from the admin dashboard; the unused full-Looker Action Hub is removed; the heap limit is 256 MB; a wrong ingest token is retried (503); PDFs that break a limit give up at once (422); and the chat that was waiting is told when a delivery gives up.

Verification

The v1.6.0 suite ran 153 tests: 142 passed, 0 failed and 11 were skipped locally. The skipped ones are MongoDB integration tests, which CI runs against a real MongoDB server, including real PDF-to-PNG conversion. A fresh scheduled email and the destination WhatsApp images are the final acceptance evidence for the live installation.

Release boundary

This report describes the implemented pilot and its remaining operational limits. It does not promise uninterrupted delivery, certify recipient read receipts, or claim a successful online dependency audit when the registry was unavailable.

Source of truth: versioned implementation, configuration and RISKS_AND_LIMITATIONS.md. The usage guide and handover provide step-by-step operator instructions.

Release scope

This document covers the routing, retry, command-replay, session-storage and PDF-resource controls, the admin and chat dashboards, the durable retry/dead-letter layer, the v1.3.1-v1.5.0 hardening (terminal dead letters, one retry schedule, log redaction, key-store batching, WhatsApp caches, LID support, shared PDF storage, dashboard retry and keep-awake) and the v1.6.0 changes (open commands, chat dashboard, QR linking only in the admin dashboard, Action Hub removal, 256 MB heap, retryable token mismatch, permanent PDF rejections and delivery-failure notes to the affected chat). The v1.6.0 suite ran 153 tests: 142 passed, 0 failed and 11 were skipped locally; the skipped ones are MongoDB integration tests, which CI runs against a real MongoDB server, including real Poppler conversion and HTTP ingestion. Tests use controlled substitutes for Gmail and WhatsApp; they do not certify delivery through the user's live Gmail, Render, MongoDB and WhatsApp accounts.

Exact package pins and a lockfile are committed. CI runs `npm audit --omit=dev --audit-level=high`; the v1.3.1 and v1.4.0 CI runs reported 0 vulnerabilities. That is a point-in-time result, not a guarantee for later dates.

Delivery reliability controls

- Delivery identity includes the Gmail message ID and destination chat. One ingested email may reach several distinct chats, with independent completion records.
- Active routes are selected before destination deduplication. A paused alias cannot conceal an active alias for the same chat.
- MongoDB uses leased claims with ownership tokens. Only a confirmed delivered record receives a duplicate acknowledgement; a busy lease remains retryable with HTTP 503.
- Failed or abandoned processing can be reclaimed. The supplied deployment lease is ten minutes; the code fallback without configuration is fifteen minutes.
- Accepted pages are checkpointed. A later-page retry resumes at the first unsaved page, and completed destinations are skipped.
- A failed destination does not prevent other available destinations from completing. Claim-lookup errors release earlier acquired claims.
- HTTP disconnects keep their processing slot until work ends, preventing callers from bypassing concurrency limits by closing the connection.

WhatsApp sending and MongoDB progress cannot be committed as one transaction. If WhatsApp accepts a page but its acknowledgement or database checkpoint is lost, retry can repeat that page. The bot does not verify recipient delivery/read receipts. The count saved means the send call completed, not that a person opened the image.

A failed or abandoned delivery is no longer solely dependent on the upstream Gmail bridge resending the source email; see "Delivery retries and dead-lettering" below for the background worker that retries it directly.

Deduplication does not merge separate Gmail message IDs. If a schedule sends separate copies to multiple aliases for the same chat, repeated-looking images can be expected. Use one alias per chat per schedule. Delivery records expire after ninety days; replay protection is bounded by retention.

Gmail and scheduling limits

The bridge polls every five minutes. A configured cutoff prevents historical backlog replay, and a missing cutoff is initialized to now. Preserve the cutoff on upgrades. Mail earlier than the cutoff is intentionally excluded.

The default query has a seven-day lookback and scans up to five hundred threads in pages of fifty. The cap is configurable from fifty to two thousand; the lookback can be one to thirty days. Extremely busy mailboxes can hide older eligible threads beyond the configured cap. Apps Script execution and service quotas can also interrupt large runs.

The bridge retains two thousand processed IDs in twenty Script Properties. Gmail labels are informational at the thread level and are not used as an exclusion filter. The MongoDB ledger provides additional duplicate protection when an older Gmail ID leaves the script's bounded ledger.

Network errors, HTTP 408/429 and 5xx are retryable. Since v1.6.0 a wrong or missing ingest token answers 503 with `Retry-After: 300`, so reports sent while the service and Apps Script tokens disagree are retried every run until the tokens match (within the bridge's lookback). Other 4xx responses are recorded as terminal rejections. Correcting a sender or route after a terminal rejection does not automatically replay that email. Likewise, paused-only mail is acknowledged and skipped; resuming a route does not replay paused mail. Use a fresh controlled scheduled delivery after correcting configuration.

Looker Studio controls PDF contents, selected pages, filters, data freshness and schedule timezone. A bot route name does not create or select a Looker report. Schedule creators may receive their own email copy. A dedicated reporting account is the appropriate operational arrangement when personal inbox copies are unwanted.

Access and session security

The ingestion endpoint requires a bearer token and checks the approved sender list, defaulting in application startup to `data-studio-noreply@google.com`. A forwarded `From` field is metadata, not independent cryptographic proof of the sender. Security depends on the trusted mailbox/bridge, ingestion token and secret alias as well as the allowlist.

Route tokens are random and stored as HMACs with a stable pepper. Exposing an alias can allow report injection through the mailbox path if other controls are compromised. Treat aliases as secrets, rotate exposed routes and replace the old schedule recipients. A changed pepper invalidates all existing aliases; never regenerate it on routine deployment.

Anyone in a chat can use every report command for that chat; a command only affects the chat it was sent in. There is no group-admin check (removed in v1.6.0 with `STUDIO_ROUTE_ADMIN_IDS` and `STUDIO_ALLOW_PUBLIC_SETUP`), so any member can create, pause, rotate or remove that chat's routes. Default route quota is twenty per chat, with serialized creation inside one process. Multiple application instances would need a database-level quota mechanism. Deletion requires an explicit `--confirm` command.

WhatsApp commands have ID/timestamp checks, stale-socket guards, in-memory deduplication and a persistent MongoDB claim ledger. The default maximum command age is twenty-four hours; persistent claims expire after seven days. A durable claim is made before executing the command, so an interrupted command can require the operator to send a fresh command with a new message ID.

Signal/WhatsApp encryption material is kept out of logs. Before v1.4.0, `libsignal` (inside Baileys) logged whole session objects, including ratchet private keys, on routine events such as "Closing session", and they appeared in the Render log viewer. `logRedaction.js` now drops that routine chatter and redacts any logged object that carries key material. Logs written before v1.4.0 may still contain such keys; restrict log access, and re-link the WhatsApp device if exposure is a concern (Signal sessions ratchet forward, so older keys lose value over time). `WA_SIGNAL_DEBUG=true` restores the routine messages for debugging, still redacted. Raising `WA_LOG_LEVEL` enables Baileys' own logger, which is not covered by this redaction.

Session records can be encrypted with AES-256-GCM using `WA_AUTH_ENCRYPTION_KEY`. Existing plaintext records migrate on read after the key is configured. Encryption is optional for backward compatibility, so deploying the code alone does not prove the hosted key is set. Preserve and back up the original key

securely. Wrong or missing keys fail to decrypt encrypted records and can require controlled session recovery/relinking.

WhatsApp is linked only from the admin dashboard, behind `STUDIO_ADMIN_TOKEN`; the separate QR setup page and its token were removed in v1.6.0. QR codes are sensitive, short-lived link credentials. They must not be committed to GitHub or copied into shared reports. The current runtime can still print linking QR codes in private operator logs, so access to logs must be restricted.

Admin dashboard

The `/admin/` dashboard and its `/admin/api/*` endpoints require an independent bearer token (`STUDIO_ADMIN_TOKEN`), rate-limited and compared with the same timing-safe check used elsewhere. It is deliberately separate from `STUDIO_INGEST_TOKEN`, the secret Apps Script uses to post reports. The admin dashboard is also the only place to link WhatsApp; while unlinked it reloads the QR code every 4 seconds.

The dashboard is a thin client over the existing `routeService/studioStore` methods used by the WhatsApp commands - it does not add a second source of truth, a database session, or server-side cookies. The token lives only in the browser (`sessionStorage`, or `localStorage` if the operator opts in) and is sent as a bearer header; it is never logged or persisted server-side. As with the WhatsApp flow, route addresses are returned once at creation/rotation time and are never stored or re-displayed in plaintext - the dashboard cannot show an existing route's address, only issue a new one via rotation.

The dashboard can also requeue a dead-lettered delivery (`POST /admin/api/deliveries/retry`) while its PDF is still stored. Listings expose the Gmail message ID and whether a retry is possible, never the PDF bytes or internal references.

This is still a shared-secret model, not per-operator accounts: anyone with the token has full route management for every chat the bot serves, and the token cannot be scoped or individually revoked without rotating it for all operators. A company rollout wanting per-person audit trails or scoped access needs real authentication (e.g. OIDC/SSO) in front of `/admin/`, which is out of scope for this release.

Chat dashboard

`/chat/` and its `/chat/api/*` endpoints log in with a WhatsApp chat ID only, sent as the `x-chat-id` header. Every request is limited to that one chat and rate-limited per client address (`STUDIO_ADMIN_RATE_LIMIT_PER_MINUTE`, default 60). Tests check that one chat cannot see or change another chat's routes or deliveries.

A chat ID is not a secret: it is shown by `!chatid`, and a personal chat's ID contains the phone number. Anyone who learns it can, for that chat, list report names and statuses, see recent deliveries (email subject, status and last error), create, pause, resume, rotate and remove routes, and retry deliveries that gave up. Removing or rotating a route stops the matching Looker Studio schedule from delivering until it is updated. A new route's address could also be added to any Looker Studio schedule, whose email comes from the approved sender, so it could deliver unrelated report pages into that chat. This is an accepted risk, chosen by the owner for simplicity. Stronger options (a per-chat secret, or real authentication) are not implemented.

Delivery retries and dead-lettering

Report processing still happens inline in the HTTP ingest request for the common case - this keeps latency low and keeps the Apps Script response contract (the documented 403/404/413/422/429/502/503 meanings; a token mismatch answers 503 since v1.6.0). What changed is what happens when that inline attempt does not finish cleanly.

The source PDF is stored once per email in the `studio_pdfs` collection before any destination is claimed, and each destination's delivery record references it. It is deleted once no pending, failed or dead-lettered delivery needs it, with a 30-day TTL index as a backstop. (Up to v1.4.0 each destination stored its own copy on its delivery record; such records remain readable.)

Before v1.5.0, the worker received that stored PDF from MongoDB as a BSON Binary object, which the PDF renderer rejects, so every background retry failed and deliveries only recovered when the Gmail bridge re-sent the email. v1.5.0 reads stored PDFs as Node Buffers (`promoteBuffers`), and integration tests now use a renderer stand-in that rejects anything else. A background worker inside the same process (`studioDeliveryWorker.js`) runs on a fixed interval (`STUDIO_DELIVERY_WORKER_INTERVAL_MS`, default 60 seconds) and picks up: deliveries left in `failed` status once their backoff has elapsed, and deliveries stuck in `processing` past their lease - the signature of a process that crashed or was redeployed mid-send. Neither case depends on Apps Script or the Gmail bridge resending the source email; the stored PDF is enough to retry independently.

Retries use exponential backoff from `STUDIO_DELIVERY_RETRY_BASE_MS` (default 60 seconds, doubling each attempt, capped at 30 minutes) and are bounded by `STUDIO_DELIVERY_MAX_ATTEMPTS` (default 6, counting the original attempt). Both the worker and upstream bridge retries honour the same backoff window, and a WhatsApp disconnect in the middle of sending does not consume an attempt (at most 24 such interruptions per delivery, after which failures count normally). A delivery that exhausts its attempts moves to `dead_letter`: it is no longer retried automatically, and the ingest endpoint answers 422 for it so the Gmail bridge stops re-uploading the PDF. A PDF that breaks a limit (more than `STUDIO_MAX_PAGES` pages, too large, bad geometry or pixel count, or not a PDF) fails the same way on every attempt, so since v1.6.0 it is marked permanent: the delivery goes straight to `dead_letter` and ingest answers 422 instead of using six attempts. It keeps its PDF for `STUDIO_DEAD_LETTER_RETENTION_MS` (default 7 days) so **Retry** in either dashboard can restore a fresh attempt budget once the cause is fixed. When a delivery is dead-lettered, the chat that was waiting gets a short note with the reason (limited to 10 per 10 minutes; skipped while WhatsApp is down; always logged). A note sent over WhatsApp cannot report a WhatsApp outage, so external monitoring of `/readyz` is still advisable.

What this still does not provide: a separate worker process or dyno (it runs in the same Node process as the HTTP server and the WhatsApp client, so a full service crash pauses both the API and the retry sweep until Render restarts it, at which point the worker resumes from MongoDB's persisted state), true concurrent workers (Render's free/starter tier runs one instance, and the design assumes that), a dead-letter **queue** in the message-broker sense (it is a status on the existing delivery record, not a separate durable queue with its own consumers), or delivery ordering guarantees across retries.

PDF and resource controls

The supplied defaults limit PDF input to 15 MiB, reports to five pages, rendered images to 7 MiB each, each image dimension to 2,400 pixels and total output to twenty million pixels. PDF metadata is checked before conversion. Documents beyond page or geometry limits are rejected rather than truncated, and the rejection is permanent (no retries). DPI is reduced within supported bounds when necessary.

Poppler inspection/conversion has timeouts and bounded command output. PNG signature, dimensions and file size are checked before use; temporary files are removed after processing. These controls reduce resource abuse but do not provide antivirus scanning, content moderation or an independent sandbox for malicious PDFs. Keep the operating system and Poppler patched.

Authenticated Studio requests are limited to thirty per minute and two concurrent requests in the supplied configuration. The HTTP body cap is 22 MiB. Base64 expansion means the HTTP cap must be considered alongside the PDF cap. Large or slow reports can still exhaust a small free-tier service; increasing limits needs capacity testing. The Node heap limit was raised from 128 MB to 256 MB in v1.6.0 after out-of-memory crashes every few days; Render Free has 512 MB in total, so the heap, Poppler and native memory still share a small budget. An hourly `Memory: heap ... log` line helps spot slow growth before it

becomes a crash.

Only the first PDF attachment is processed by the current email service. The supported workflow is one scheduled report PDF per email. The renderer sends images, not interactive dashboards, clickable charts or the original PDF attachment.

Platform and production limits

Baileys is an unofficial WhatsApp integration and its pinned package is a release candidate. WhatsApp protocol or account changes can interrupt service. Reconnect backoff and persisted sessions reduce disruption, but do not establish a service guarantee. Official Business Platform suitability must be evaluated against the actual chat/destination requirements.

Render's free tier spins a web service down after 15 minutes without inbound traffic; while it sleeps, the retry worker and WhatsApp connection are paused. The Gmail bridge's keep-awake ping (Script Property `KEEP_SERVICE_AWAKE`, default true) calls `/healthz` every five minutes to prevent that. Render grants 750 free instance hours per workspace per month and suspends all free web services when they run out; one always-on service uses at most 744, so turn the ping off if other free services share the workspace. Deploys in the current Render service are manual unless Auto-Deploy is enabled in its settings.

The Render free-tier Blueprint, Apps Script polling and MongoDB availability form a pilot arrangement. Cold starts, platform outages, resource limits, revoked account access and disconnected linked devices can delay reports. `/healthz` only proves HTTP liveness; `/readyz` proves current WhatsApp connection state; `/versionz` identifies the running code. None proves an individual report was delivered.

Report processing still starts inline inside the HTTP request, but a failed or interrupted delivery is now retried by an in-process background worker with backoff and a bounded attempt count, ending in a terminal `dead_letter` status rather than being lost or retried forever - see "Delivery retries and dead-lettering" above. This is not the same as an independent worker process, a message-broker-style queue, or horizontally scaled workers: it is one worker loop inside the same Node process as the HTTP server, backed by MongoDB for durability across restarts. The Gmail bridge separately supplies its own retries within its cutoff/lookback/scan bounds for cases the delivery worker cannot help with (for example, a route that did not exist yet when the email first arrived). Production volume at real scale still benefits from a dedicated queueing system, always-on multi-instance capacity, external monitoring, and tested recovery drills - none of which this release adds. Delivery-failure notes to the affected chat (v1.6.0) cover the most common "a report silently did not arrive" case.

Company rollout also needs owned service identities, secret management, backup/restore exercises, account recovery, destination/access reviews, confidential-data handling, retention rules and incident response. No paid infrastructure, new company accounts or official WhatsApp migration is silently included in this release.

Acceptance and recovery

Verify the live version/commit, deployed secrets/configuration and preserved Apps Script source/cutoff. Schedule a fresh small report after the cutoff to a controlled chat, then verify the PDF email, forwarding log and every WhatsApp page. Test a second chat, busy/stale claims, later-page retry, pause/resume and restart recovery before broader use.

When diagnosing missing delivery, correlate the Gmail message ID across Apps Script and service logs. Do not delete all routes, aliases or the entire processed ledger as a general recovery step. Correct the specific error, preserve valid aliases for other chats and use a new controlled delivery where appropriate.

The regenerated PDF is a readable companion to this file. The source code, configuration files and recorded release checks remain the implementation evidence. See `USAGE_GUIDE.md` and `LOOKER_INTEGRATION_HANDBOOK.md` for operator steps.